

UHF SDK

`com.rockchip.uhf:uhf-sdk`

Developer Documentation

Version 1.0.0

Implements UHF-SRS-001 §7 (SDK-01 through SDK-10)

Target platform: rockchip/rk3576/rk3576_u, AOSP 14 (API 34)

Generated 2026-09-18

1. Overview

The UHF SDK is a thin Kotlin/Java facade over the platform's `android.hardware.uhf.UhfManager` (UHF-SRS-001), distributed as `com.rockchip.uhf:uhf-sdk` so third-party apps can build against it without platform source or `@SystemApi` access.

It adds no protocol logic of its own (SDK-03): every method on `UhfSdkSession` is a 1:1 delegation to the matching `UhfSession` method, verified by direct comparison against the compiled platform class. This is also why a platform-only wire-protocol bug fix never forces an SDK version bump (SDK-07) — there is no protocol code here to fix.

What's included

- `UhfSdk` — static entry points: `isSupported()`, `open()`, `openFake()`
- `UhfSdkSession` — the full session API surface, as an interface
- `FakeUhfSdkSession` — an in-memory simulated reader for offline development (SDK-09)
- `UhfSdkKt.kt` — Kotlin coroutine (`inventorySuspend`) and Flow (`tagFlow`) adapters (SDK-08)
- `sample` app — a reference Activity exercising every API group (SDK-05)

2. Integration Guide

2.1 Permissions

Declare in your app's manifest:

```
<uses-permission android:name="android.permission.UHF_READ" />
<!-- Only if your app writes tags: -->
<uses-permission android:name="android.permission.UHF_WRITE" />
```

`UHF_READ` is *normal* (granted at install, no runtime prompt). `UHF_WRITE` is *dangerous* — request it at runtime the same way you would `CAMERA` or `RECORD_AUDIO`. Region, RF power and antenna configuration (`UHF_CONFIGURE`) are *signature|privileged* — not requestable by a normal app at all; those calls fail with `UhfException(ERROR_BUSY, ...)` for anything but a platform/OEM-signed caller. An app that needs `UHF_CONFIGURE` is an OEM/provisioning tool, not a third-party consumer of this SDK.

Always call `UhfSdk.isSupported(context)` before `UhfSdk.open(context)` — it never throws, on any device. `open()` on an unsupported device throws `UhfException(ERROR_DEVICE_LOST, ...)` rather than returning null, so skipping the check fails loudly in one place instead of a `NullPointerException` three calls later.

2.2 The antenna/throughput tradeoff

Measured on the reference hardware (UHF-SRS-001 §2.4): streaming reads/sec roughly **halves with each additional antenna port enabled** (130.9 reads/sec at one port, 51.9 at four). If your application only needs read range, not multi-port coverage, `setEnabledAntennas(ANTENNA_1)` before streaming is the single highest-leverage performance decision available.

2.3 The RF power ceiling

`setRfPower()` refuses anything above the platform's safe ceiling (default 20/30) unless you pass `allowAboveSafeCeiling = true` explicitly. This exists because the reference hardware **browns out at 30/30** on its stock power supply — every command still answers normally, but inventory returns nothing. If inventory suddenly returns nothing after a power change, suspect the power supply before the SDK.

2.4 Offline development (SDK-09)

`UhfSdk.openFake()` returns a `UhfSdkSession` backed by an in-memory simulated reader — no device, no `@SystemApi` access, buildable and testable from a clean checkout. It enforces the same caller-visible contracts the platform does: an all-zero kill password is refused, a permanent lock needs explicit confirmation, an invalid word range is rejected — so code passing against the fake has already cleared what it will meet on real hardware. It ships 5 simulated tags by default; use `addSimulatedTag/removeSimulatedTag/clearSimulatedTags` to control the population.

3. Compatibility Matrix (SDK-07)

SDK version	Requires platform	Notes
1.0.x	UHF-SRS-001 Phase 1+, SDK 34 (AOSP 14)	Session lifecycle, identity, region, power, antennas, polled/buffered inventory, streaming (§4.1-4.9).
1.1.x (planned)	UHF-SRS-001 Phase 3	Adds tag memory access and security (§4.10-4.11) — <code>readTagMemory/writeTagMemory/killTag/lockTag/etc.</code>
—	Phase 4/5 not implemented	Read protection, EAS, vendor extensions, and GPIO/peripherals have no SDK surface yet.

The SDK is versioned independently of the platform build: a platform patch that only fixes a wire-protocol bug does not require an SDK release, because this module adds no protocol logic of its own — everything is a 1:1 delegation to `UhfManager/UhfSession`.

4. API Reference

Every signature below is transcribed directly from the module's source, cross-checked against the platform's compiled `android.hardware.uhf` classes during this SDK's build.

4.1 UhfSdk

`public final class UhfSdk` — static entry points. Not instantiable.

`static boolean isSupported(Context context)`

Never throws, on any device. Checks `FEATURE_UHF_READER`.

`static UhfSdkSession open(Context context) throws UhfException`

Opens a real session. Throws `ERROR_DEVICE_LOST` (not null) if unsupported.

`static UhfSdkSession openFake()`

Returns an offline `FakeUhfSdkSession` — no device or `@SystemApi` access needed.

4.2 UhfSdkSession

`public interface UhfSdkSession` extends `AutoCloseable` — the full session surface. Every method matches its `UhfSession` counterpart's name, parameters and thrown exception exactly.

Lifecycle

`boolean isValid()`

`void close()`

`UhfReaderInfo getReaderInfo() throws UhfException`

Region & power

`void setRegion(int bandCode) throws UhfException`

`void setRfPower(int power) throws UhfException`

`void setRfPower(int power, boolean allowAboveSafeCeiling) throws UhfException`

`int getRfPower() throws UhfException`

`int getMaxRfPower() throws UhfException`

`int getSafeRfPowerCeiling() throws UhfException`

Antennas

`void setEnabledAntennas(int mask) throws UhfException`

`int getEnabledAntennas() throws UhfException`

`void setAntennaCheckEnabled(boolean enabled) throws UhfException`

`boolean isAntennaCheckEnabled() throws UhfException`

Inventory

`List<UhfTag> inventory() throws UhfException`

`List<UhfTag> inventory(InventoryParams params) throws UhfException`

Streaming

`void setStreamParams(StreamParams params) throws UhfException`

```

void startStreaming(Executor executor, TagCallback callback) throws UhfException
void stopStreaming()
boolean isStreaming() throws UhfException
StreamStats getStreamStats() throws UhfException
void resetStreamStats() throws UhfException

```

Buffered inventory

```

void startBufferedInventory(InventoryParams params) throws UhfException
void stopBufferedInventory()
int getBufferedTagCount() throws UhfException
List<UhfTag> readBuffer() throws UhfException
void clearBuffer() throws UhfException

```

Tag memory & security

```

byte[] readTagMemory(TagFilter filter, int bank, int wordAddress, int wordCount) throws
UhfException
void writeTagMemory(TagFilter filter, int bank, int wordAddress, byte[] data) throws
UhfException
void writeEpc(TagFilter filter, byte[] newEpc) throws UhfException
void blockWrite(TagFilter filter, int bank, int wordAddress, byte[] data) throws
UhfException
void blockErase(TagFilter filter, int bank, int wordAddress, int wordCount) throws
UhfException
void setVerifyWrites(boolean verify) throws UhfException
void killTag(TagFilter filter, byte[] killPassword) throws UhfException
void lockTag(TagFilter filter, int region, int lockState, byte[] accessPassword, boolean
confirmIrreversible) throws UhfException
void setAccessPassword(TagFilter filter, byte[] oldPassword, byte[] newPassword) throws
UhfException
void setKillPassword(TagFilter filter, byte[] accessPassword, byte[] newKillPassword) throws
UhfException

```

Constants (mirrored from UhfSession)

Memory banks	BANK_RESERVED=0, BANK_EPC=1, BANK_TID=2, BANK_USER=3
Lock regions	LOCK_REGION_KILL_PW=0, LOCK_REGION_ACCESS_PW=1, LOCK_REGION_EPC=2, LOCK_REGION_TID=3, LOCK_REGION_USER=4
Lock states	LOCK_OPEN=0, LOCK_LOCKED=1, LOCK_PERM_OPEN=2, LOCK_PERM_LOCKED=3

4.3 UhfException error codes

Constant	Value
ERROR_BUFFER_FULL	1
ERROR_ANTENNA_FAULT	2

ERROR_COMMAND_REJECTED	3
ERROR_PROTOCOL	4
ERROR_INVALID_PARAMETER	5
ERROR_TIMEOUT	6
ERROR_DEVICE_LOST	7
ERROR_BUSY	8

4.4 FakeUhfSdkSession (SDK-09)

An in-memory simulated reader implementing the full `UhfSdkSession` surface. Ships 5 simulated tags by default. Enforces FR-SEC-02 (kill password may not be all-zero), FR-SEC-04 (permanent lock requires `confirmIrreversible=true`), and FR-MEM-06/07 (word-range validation) — the same contracts the real platform arbiter enforces, so code that passes against the fake has already cleared what it will meet on real hardware. Streaming is backed by a genuine `ScheduledExecutorService`, not a mock.

Test-only controls: `addSimulatedTag(UhfTag)`, `removeSimulatedTag(byte[] epc)`, `clearSimulatedTags()`.

4.5 Kotlin extensions (SDK-08)

```
suspend fun UhfSdkSession.inventorySuspend(): List<UhfTag>
```

Runs the blocking `inventory()` on `Dispatchers.IO`. Deliberately not named `inventory` — verified empirically that a same-named `suspend` extension is silently shadowed by the interface's existing blocking member and becomes unreachable through ordinary call syntax.

```
fun UhfSdkSession.tagFlow(params: StreamParams = StreamParams.Builder().build()):
    Flow<UhfTag>
```

A cold `Flow` of tag reads: starts streaming on collection, stops on cancellation/teardown. Stream errors close the flow with the originating `UhfException`.

5. Sample Usage

Adapted from the reference app (sdk/sample/), demonstrating every API group in the order UHF-SRS-001 §7's own usage example presents them.

```
if (!UhfSdk.isSupported(context)) {
    // no reader on this device
    return
}
val session = try {
    UhfSdk.open(context)
} catch (e: UhfException) {
    UhfSdk.openFake() // offline dev/test fallback (SDK-09)
}

session.setEnabledAntennas(0b0001) // fewer ports = faster reads
session.setRfPower(20) // stays under the safe ceiling

val tags = session.inventorySuspend()
session.tagFlow()
    .distinctUntilChangedBy { it.epcHex }
    .onEach { tag -> Log.i(TAG, "${tag.epcHex} rssi=${tag.rssi}") }
    .launchIn(scope)

val filter = TagFilter.byEpc(tags.first().epc)
session.writeTagMemory(filter, UhfSdkSession.BANK_USER, 0, byteArrayOf(0x12, 0x34))
val readBack = session.readTagMemory(filter, UhfSdkSession.BANK_USER, 0, 1)

session.close()
```

6. What's Verified vs. What Isn't

Unlike most of the platform patch series (checked by reading only, no build toolchain available), this module's Java and Kotlin source was **actually compiled**, and for the fake reader, **actually run**:

- UhfSdk, UhfSdkSession, RealUhfSdkSession, FakeUhfSdkSession compile cleanly against the platform's real compiled android.hardware.uhf classes (not a hand-written stub) — confirmed twice: once against a frozen system-API stub jar, and again against the actual framework.jar produced by a full, successful platform build.
- UhfSdkKt.kt compiles with a real Kotlin 1.8.10 compiler against a real kotlincoroutines-core jar.
- FakeUhfSdkSessionTest's 12 tests were compiled **and executed** against JUnit 4, all passing — including real background streaming via a genuine ScheduledExecutorService.
- The sample app and UhfTestApp (a separate, plain-Soong platform test app) both compile cleanly; UhfTestApp was additionally installed and run on real hardware.

Not yet done:

- No Gradle/AGP build of the SDK module itself — unavailable in every environment this project has been developed in.
- The AAR distributed alongside this documentation was hand-assembled (real javac/kotlinc output, packed into the standard AAR zip layout manually) rather than produced by `gradlew assembleRelease` — functionally equivalent for a resource-less library, but missing whatever AGP-specific metadata that task would add.

- No device or emulator run of the sample app.
- SDK-02's platform stub jar (generated from `api/system-current.txt` via `metalava`) was not built as a standalone artifact; verification instead used the real compiled platform classes directly.